Aula-Complemento: NP-Completude Reduções, Intuição e Prática

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

Análise de Algoritmos – 2026.1

Roteiro

1. Retomada Rápida
2. NP-Completude: Definição e Intuição
3. Redução Polinomial
4. Exemplos Detalhados
5. Erros Comuns
6. Exercícios
7. Gabarito
8. Fechamento

De "NP" para "NP-Completo"

O que já sabemos

- P: problemas **resolvíveis** em tempo polinomial. Ex.: BFS, ordenação.
- NP: problemas cuja resposta SIM tem **certificado verificável** em tempo polinomial.
- $P \subseteq NP$: todo problema em P também está em NP (provado).

Pergunta natural

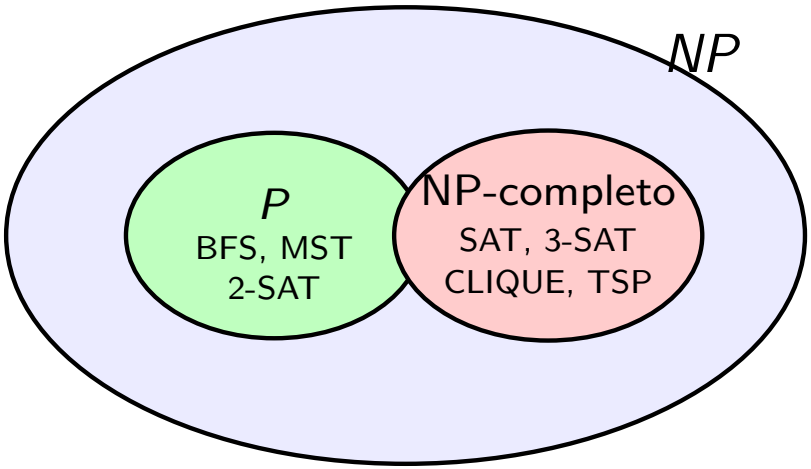
Dentro de NP, todos os problemas são igualmente difíceis?

Resposta

Não. Existe um subconjunto de problemas em NP que são os **mais difíceis de todos**: se qualquer um deles tiver solução polinomial, *todos* os problemas de NP também terão solução polinomial.

Esse subconjunto é a classe dos **NP-completos**.

Mapa das Classes de Complexidade



O que o diagrama diz

- $P \subseteq NP$: sempre.
- NP-completo $\subseteq NP$: por definição.
- NP-completo e P são disjuntos (se $P \neq NP$).

Hipótese $P = NP$

Os três conjuntos colapsariam em um só. A maioria: $P \neq NP$ (sem prova formal).

O Que é NP-Completo?

Definição

Um problema de decisão L é **NP-completo** se satisfaz **duas condições**:

1. $L \in NP$ — tem certificado verificável em tempo polinomial.
2. **Todo** problema $A \in NP$ se reduz a L em tempo polinomial ($A \leq_p L$).

O que a condição 2 significa na prática

L é pelo menos tão difícil quanto *qualquer* problema em NP. Resolver L em tempo polinomial resolveria **todos** os problemas de NP — incluindo SAT, CLIQUE, TSP, Ciclo Hamiltoniano...

Consequência imediata

$$\text{Algoritmo polinomial para } L \Rightarrow P = NP$$
$$P \neq NP \text{ (acreditado)} \Rightarrow \text{nenhum NP-completo está em } P$$

Analogia: O Tradutor Universal

Cenário

Você tem um problema A difícil de resolver diretamente. Mas existe um "oráculo" que resolve o problema B instantaneamente.

Estratégia com redução

1. **Traduza** uma instância de A em uma instância de B (em tempo polinomial, preservando SIM/NÃO).
2. **Consulte** o oráculo para B.
3. **Retorne** a resposta — que é a mesma para A.

Conclusão

Se A é difícil e você consegue fazer essa tradução, então B também é difícil.

Escrevemos: $A \leq_p B$ ("A se reduz polinomialmente a B").

Intuição: "B é pelo menos tão difícil quanto A."

Redução Polinomial: Definição e Direção

Definição

$A \leq_p B$ significa: existe função f, computável em tempo polinomial, tal que

$$x \in A \iff f(x) \in B.$$

f transforma instâncias de A em instâncias de B **preservando SIM/NÃO**.

A direção importa!

Para provar que B é difícil,
reduza um **já difícil** para B:

$$A_{\text{difícil}} \leq_p B.$$

Não confunda com $B \leq_p A$.

Para provar NP-completude

1. Mostre $B \in NP$.
2. Escolha NPC A conhecido.
3. Construa f polinomial.
4. Prove $x \in A \iff f(x) \in B$.

Receita de Prova de NP-Completude

Passo 1 — Mostre que $L \in NP$

Descreva um **certificado** de tamanho polinomial e um **verificador** polinomial. Sem esse passo, não pode chamar o problema de NP-completo.

Passo 2 — Escolha o problema fonte

Use um NP-completo já conhecido. SAT e 3-SAT são as escolhas mais comuns; CLIQUE e CICLO-HAMILTONIANO também são usados.

Passo 3 — Construa a transformação f

Dado qualquer instância x do fonte, construa $f(x)$ instância de L, em tempo polinomial.

Passo 4 — Prove a equivalência **nos dois sentidos**

- ($\Rightarrow$) Se x é SIM para o fonte $\Rightarrow$ $f(x)$ é SIM para L.
- ($\Leftarrow$) Se $f(x)$ é SIM para $L \Rightarrow$ x é SIM para o fonte.

Erro mais comum

Provar só "$\Rightarrow$" e esquecer "$\Leftarrow$". A equivalência **exige os dois sentidos**.

Exemplo 1 — SAT: O Primeiro NP-Completo

Contexto (Cook–Levin, 1971)

SAT foi o **primeiro** problema provado NP-completo. Todo problema de NP pode ser reduzido a SAT — isso é o que o teorema de Cook prova.

Verificação concreta

$\phi = (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z)$, certificado: $x{=}V$, $y{=}F$, $z{=}V$.

1. $(V \vee F) = V$ **OK**
2. $(F \vee V) = V$ **OK**
3. $(V \vee F) = V$ **OK**

Todas as cláusulas satisfeitas $\Rightarrow$ certificado **válido**. Custo: $O(m \cdot n)$.

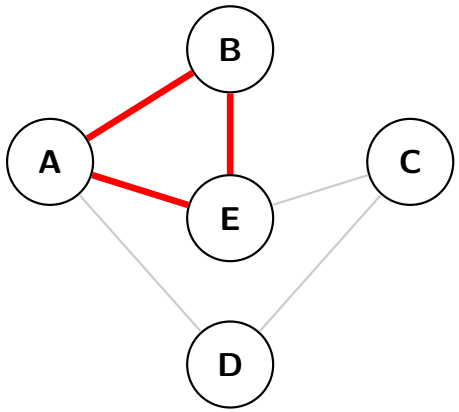
Por que SAT é estratégico?

Grande parte das provas de NP-completude reduz SAT (ou 3-SAT) ao novo problema. 3-SAT (cláusulas com exatamente 3 literais) também é NP-completo e frequentemente mais fácil de usar como fonte de redução.

Exemplo 2 — CLIQUE: Certificado e Verificação Detalhada

Problema

Dado: grafo $G = (V, E)$, inteiro k.
Decisão: existe $C \subseteq V$ com $|C| \geq k$ tal que todo par em C tem aresta?



clique $\{A, B, E\}$, k=3

Certificado e verificador

Certificado: $C = \{A, B, E\}$. Tamanho: $O(n)$.

Verificador passo a passo:

1. $|C| \geq k$? $3 \geq 3$. **OK**.
2. $(A, B) \in E$? **sim**.
3. $(A, E) \in E$? **sim**.
4. $(B, E) \in E$? **sim**.

Custo: $O(k^2)$ verificações $\leq O(n^2)$.

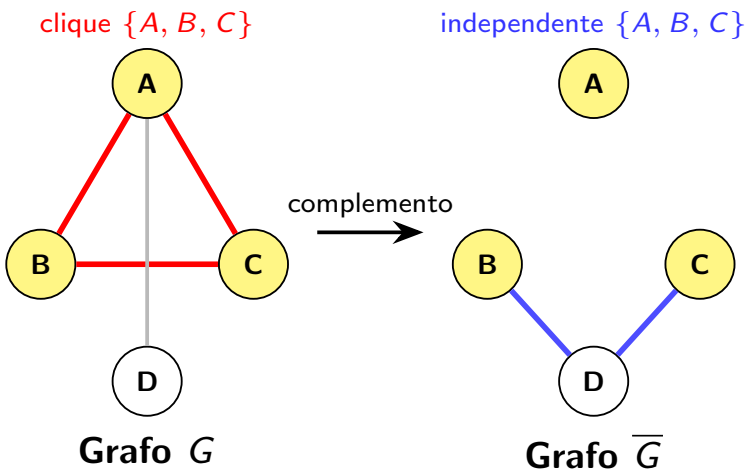
Conclusão

Certificado $O(n)$ + verificação $O(n^2)$
$\Rightarrow$ **CLIQUE** $\in NP$. ■

Exemplo 3a — CLIQUE vs. CONJUNTO INDEPENDENTE

Definições

- **CLIQUE**(G, k)**:** existe $C \subseteq V$, $|C| \geq k$, tal que **todo** par em C possui aresta?
- **INDEPENDENTE**(G, k)**:** existe $S \subseteq V$, $|S| \geq k$, tal que **nenhum** par em S possui aresta?



O que o grafo complementar faz

$\overline{G}$ **inverte** todas as arestas. Os nós $\{A, B, C\}$ formam clique em G (três arestas vermelhas) e **ficam sem nenhuma aresta** em $\overline{G}$ — tornando-se um conjunto independente.

Exemplo 3b — A Redução e as Consequências

Equivalência formal

$$C \text{ é } \textbf{clique} \text{ de tamanho } k \text{ em } G \iff C \text{ é } \textbf{independente} \text{ de tamanho } k \text{ em } \overline{G}.$$

Prova: se todo par em C tem aresta em G, então nenhum par tem aresta em $\overline{G}$ (e vice-versa). ■

As duas reduções

- CLIQUE $\leq_p$ INDEPENDENTE: dado (G, k), construa $(\overline{G}, k)$.
- INDEPENDENTE $\leq_p$ CLIQUE: dado (G, k), construa $(\overline{G}, k)$.

Construir $\overline{G}$ custa $O(|V|^2)$ — tempo **polinomial**.

Conclusão

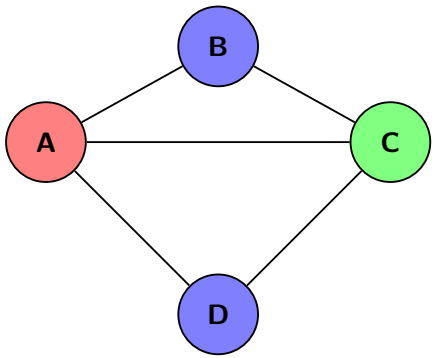
Como CLIQUE é NP-completo e INDEPENDENTE $\in NP$, a redução CLIQUE $\leq_p$ INDEPENDENTE prova que **INDEPENDENTE também é NP-completo**.

Lição: problemas de aparência oposta podem ser igualmente difíceis. Se você resolver um em tempo polinomial, resolve o outro também.

Exemplo 5 — COLORAÇÃO de Grafos

Problema

Dado: grafo $G = (V, E)$, inteiro k.
Decisão: é possível colorir os vértices com $\leq k$ cores de modo que vértices adjacentes tenham cores **diferentes**?



3-colorível: VM, AZ, VD

COLORAÇÃO $\in NP$

Certificado: função $f : V \to \{1, \ldots, k\}$. Tamanho: $O(n)$.

Verificador:

1. Para cada $v \in V$: $1 \leq f(v) \leq k$? $O(n)$.
2. Para cada $(u, v) \in E$: $f(u) \neq f(v)$? $O(|E|)$.

Custo total: $O(n + |E|)$ — polinomial. ■

Caso especial que muda tudo

2-COLORAÇÃO (IS-BIPARTITE) $\in P$ — BFS em $O(|V| + |E|)$.
k-**COLORAÇÃO** para $k \geq 3$ é NP-completo. Pequena mudança no parâmetro k, mudança enorme na dificuldade!

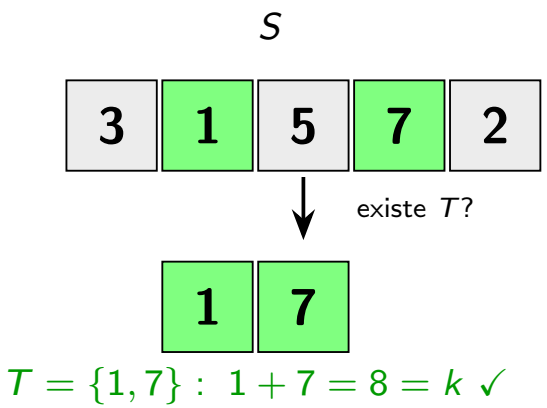
Exemplo 6a — SUBSET-SUM: O Problema e a Instância

Problema

Dado: conjunto de inteiros $S = \{s_1, \ldots, s_n\}$ e alvo k.
Decisão: existe subconjunto $T \subseteq S$ tal que $\sum_{s \in T} s = k$?

Instância: $S = \{3, 1, 5, 7, 2\}$, $k = 8$

Há $2^5 = 32$ subconjuntos possíveis. A pergunta: *existe* algum que soma 8 — sem enumerar todos?



Relevância

SUBSET-SUM é NP-completo, mas o **certificado** é trivial de verificar. Isso ilustra que em *NP* a dificuldade está em **encontrar**, não em **verificar**.

Exemplo 6b — SUBSET-SUM: Certificado e Verificador

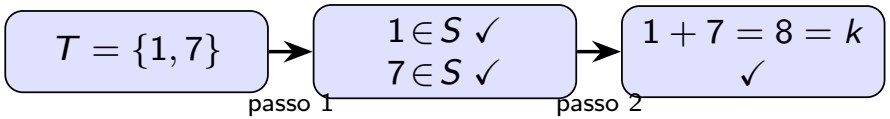
Certificado — dado ($S = \{3, 1, 5, 7, 2\}$, $k = 8$)

O certificado é o subconjunto $T = \{1, 7\}$. Tamanho: $|T| \leq |S| = n$ — **polinomial**.

Verificador passo a passo

1. **Pertinência:** para cada $t \in T$, verifique $t \in S$.
   $1 \in S$ **OK**; $7 \in S$ **OK**. Custo: $O(n)$.
2. **Soma:** calcule $\sum_{t \in T} t = 1 + 7 = 8$; compare com $k = 8$. **OK**. Custo: $O(n)$.

Custo total: $O(n)$ — polinomial.



Conclusão

Certificado de tamanho $O(n)$ + verificação em $O(n) \Rightarrow$ **SUBSET-SUM** $\in NP$. ■

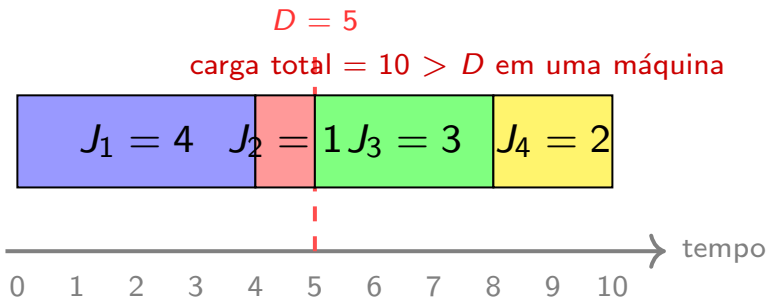
Exemplo 7a — ESCALONAMENTO em 2 Máquinas: O Problema

Problema

Dado: n tarefas com tempos $p_1, \ldots, p_n$, duas máquinas e prazo D.
Decisão: é possível atribuir cada tarefa a uma máquina de modo que nenhuma ultrapasse D no total?

Instância: J_1=4, J_2=1, J_3=3, J_4=2, $D = 5$

Soma total $= 10$. É possível dividir em dois grupos de soma ≤ 5 cada?



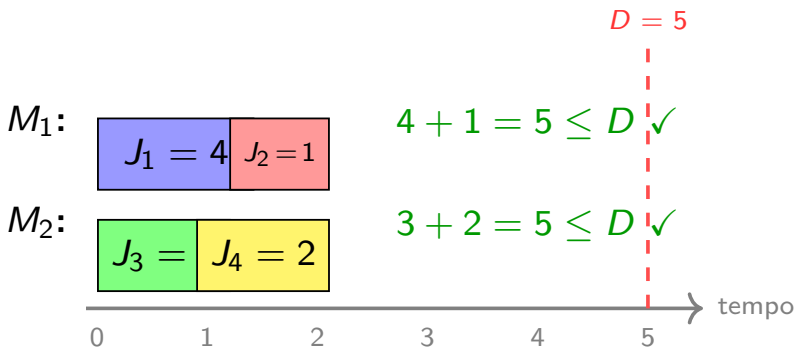
Ligação com PARTITION

Distribuir tarefas em 2 máquinas com prazo D é equivalente a particionar os tempos em dois grupos de soma $\leq D$ — mesma estrutura de PARTITION e SUBSET-SUM.

Exemplo 7b — ESCALONAMENTO: Certificado e Verificação

Certificado: atribuição $f : \{J_1, J_2, J_3, J_4\} \to \{M_1, M_2\}$

$M_1 \leftarrow \{J_1, J_2\}$, $M_2 \leftarrow \{J_3, J_4\}$. Tamanho: n bits — **polinomial**.



Verificador passo a passo

1. Toda tarefa atribuída a exatamente uma máquina? **OK**. Custo: $O(n)$.
2. $\sum_{J_i \in M_1} p_i = 4 + 1 = 5 \leq D = 5$? **OK**. Custo: $O(n)$.
3. $\sum_{J_i \in M_2} p_i = 3 + 2 = 5 \leq D = 5$? **OK**. Custo: $O(n)$.

Conclusão

Certificado de tamanho $O(n)$ + verificação em $O(n)$ $\Rightarrow$
ESCALONAMENTO-2-MÁQUINAS $\in NP$. ■

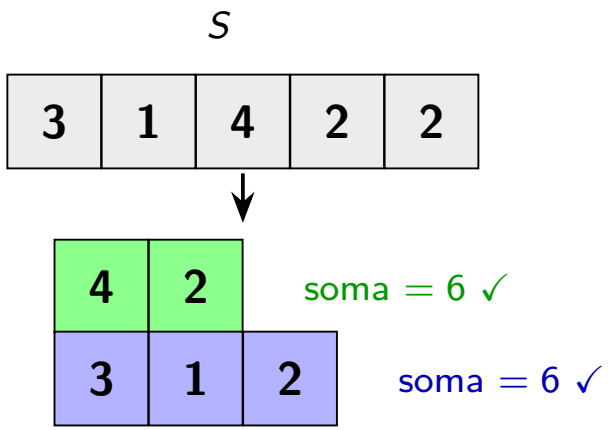
Exemplo 8a — Partição: O Problema

Problema

Dado: conjunto de inteiros S.
Decisão: existe $S_1 \subseteq S$ tal que $\sum_{s \in S_1} s = \sum_{s \in S \setminus S_1} s$? (i.e. $\sum S_1 = \frac{\sum S}{2}$)

Instância: $S = \{3, 1, 4, 2, 2\}$, $\sum S = 12$, meta $= 6$

É possível dividir S em dois grupos de soma igual?



Intuição

"Dividir a conta ao meio": as duas partes devem somar o mesmo valor.
O problema é NP-completo — decidir se isso é possível é tão difícil quanto SAT.

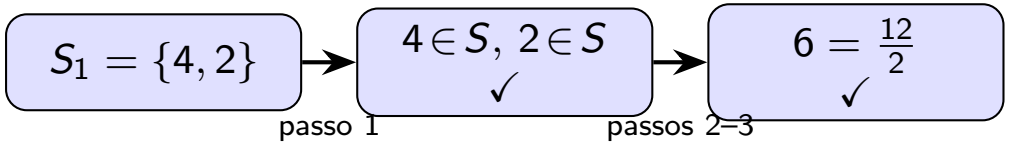
Exemplo 8b — Partição: Certificado, Verificador e NP-Completude

Certificado: $S_1 = \{4, 2\}$. Tamanho: $O(n)$.

Verificador passo a passo — dado (S, S_1)

1. Para cada $s \in S_1$: verifique $s \in S$. **OK**. Custo: $O(n)$.
2. Calcule $\text{total} = \sum S = 12$. Custo: $O(n)$.
3. Calcule $\sum S_1 = 4 + 2 = 6 = \frac{12}{2}$. **OK**. Custo: $O(n)$.

Custo total: $O(n)$ — polinomial.



Conclusão e NP-Completude

Certificado $O(n)$ + verificação $O(n) \Rightarrow$ **PARTIÇÃO** $\in NP$.

Partição$(S) \equiv$ SUBSET-SUM$(S, \frac{\sum S}{2})$. Como SUBSET-SUM $\leq_p$ PARTIÇÃO e PARTIÇÃO $\in NP$, **PARTIÇÃO é NP-completo**. ■

Exemplo 4a — Redução: SUBSET-SUM $\leq_p$ PARTITION

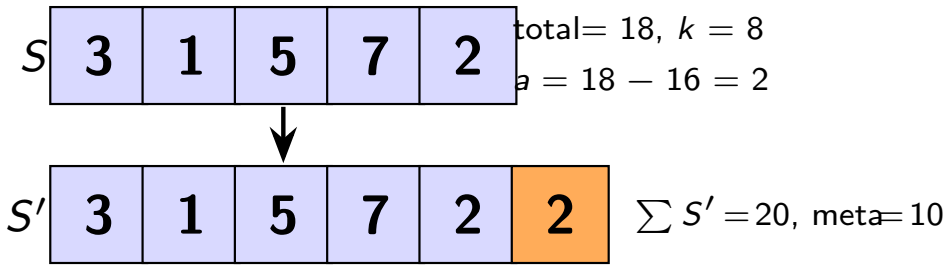
Os dois problemas

SUBSET-SUM(S, k)**:** existe $T \subseteq S$ com $\sum T = k$?

PARTITION(S')**:** existe $S_1 \subseteq S'$ com $\sum S_1 = \frac{\sum S'}{2}$?

Transformação em $O(n)$: dado (S, k), crie $S' = S \cup \{a\}$

Calcule $\text{total} = \sum S$. Adicione $a = \text{total} - 2k$. **Ex.:** $S = \{3, 1, 5, 7, 2\}$, $k = 8$, total$= 18$, $a = 2$. $\sum S' = 20$, meta$= 10$.



Intuição

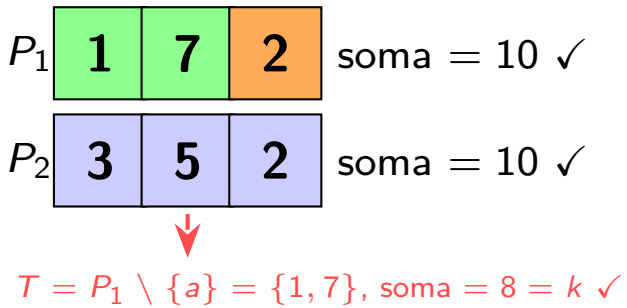
a é calibrado para que toda partição equilibrada de S' force um dos grupos a corresponder exatamente ao subconjunto de S que soma k.

Exemplo 4b — SUBSET-SUM $\leq_p$ PARTITION: Prova da Equivalência

($\Rightarrow$) $T = \{1, 7\}$ é solução de SUBSET-SUM (soma $= 8 = k$)

Defina $P_1 = T \cup \{a\} = \{1, 7, 2\}$ e $P_2 = S \setminus T = \{3, 5, 2\}$.

$$\sum P_1 = 1+7+2 = 10 \;=\; \sum P_2 = 3+5+2 = 10. \quad \checkmark$$



($\Leftarrow$) Se (P_1, P_2) é partição de S' com $a \in P_1$...

$T = P_1 \setminus \{a\}$ soma $\frac{\sum S'}{2} - a = 10 - 2 = 8 = k$. Logo $T \subseteq S$ é solução de SUBSET-SUM. ■

Custo: construir S' custa $O(n)$. Portanto **SUBSET-SUM $\leq_p$ PARTITION**.

Erros Clássicos — e Como Evitá-los

Erro	**Por que está errado**
"NP = não polinomial"	NP significa *verificável* em tempo polinomial. Problemas em P também estão em NP.
"Mostrei ($\Rightarrow$), já basta"	A equivalência exige **ambos** os sentidos: $\Rightarrow$ e $\Leftarrow$.
"Fiz $B \leq_p A$ para provar que B é difícil"	Para mostrar B difícil, precisa de $A_{\text{NPC}} \leq_p B$, não o contrário.
"Decidi não provar $B \in NP$"	Sem provar $B \in NP$, você não pode chamar B de NP-completo.

Regra de ouro

$$L \in NP \quad + \quad A_{\text{NPC}} \leq_p L \quad \Rightarrow \quad L \text{ é NP-completo.}$$

E Quando o Problema é NP-Completo na Prática?

Alternativas ao algoritmo exato polinomial

- **Heurísticas:** boas soluções rápidas, sem garantia de ótimo.
  Ex.: algoritmo guloso para coloração, vizinho mais próximo para TSP.
- **Algoritmos de aproximação:** solução com garantia de qualidade.
  Ex.: algoritmo de Christofides para TSP métrico devolve tour $\leq 1{,}5\times$ o ótimo.
- **Branch-and-Bound / Cut:** exato com podas inteligentes; escalável para instâncias médias.
- **Parametrização (FPT):** eficiente quando algum parâmetro do problema é pequeno.
- **SAT / ILP solvers:** ferramentas industriais que exploram estrutura de instâncias reais.

Mensagem

NP-completo não é sinônimo de "insolúvel".
É um sinal de que a abordagem deve ser **adequada ao contexto**: tamanho da instância, qualidade exigida e tempo disponível.

Exercícios

Exercício 1 — Certificado e Verificador

Mostre que **SUBSET-SUM** $\in NP$.
Dado $S = \{s_1, \ldots, s_n\}$ e inteiro k: existe $T \subseteq S$ com $\sum_{t \in T} t = k$?

(a) Descreva o certificado e justifique o tamanho.
(b) Descreva o verificador passo a passo e calcule o custo assintótico.
(c) Conclua formalmente que SUBSET-SUM $\in NP$.

Exercício 2 — Classificação Fundamentada

Classifique cada item e justifique em **uma frase**:

(a) 2-SAT (b) 3-SAT (c) "Retornar o menor tour do TSP"

Exercício 3 — Partição $\in NP$ e NP-Completude

Dado $S = \{5, 3, 2, 4, 2\}$ (soma total $= 16$, meta $= 8$):

(a) Descreva o certificado e o verificador para PARTIÇÃO. Qual o custo assintótico?
(b) Por que PARTIÇÃO pode ser visto como caso especial de SUBSET-SUM?
(c) Sabendo que SUBSET-SUM $\leq_p$ PARTIÇÃO, o que se conclui sobre PARTIÇÃO?

Gabarito — Exercício 1: SUBSET-SUM $\in$ *NP*

(a) Certificado

O certificado é o subconjunto $T \subseteq S$ tal que $\sum_{t \in T} t = k$.
Tamanho: $|T| \leq n$ — **polinomial**.

(b) Verificador — dado (S, k, T)

1. Para cada $t \in T$: verifique $t \in S$. Custo: $O(n)$.
2. Calcule $\sum_{t \in T} t$ e compare com k. Custo: $O(n)$.

Custo total: $O(n)$ — polinomial.

(c) Conclusão

Certificado de tamanho $O(n)$ + verificação em $O(n) \Rightarrow$ **SUBSET-SUM** $\in$ *NP*. ■

Gabarito — Exercício 2: Classificação

(a) 2-SAT $\in P$

Resolvível em $O(n+m)$ via Componentes Fortemente Conexas (algoritmo de Kosaraju ou Tarjan). A restrição de exatamente 2 literais por cláusula torna o problema tratável.

(b) 3-SAT é NP-completo

Apesar da restrição de 3 literais por cláusula, o problema permanece NP-completo. Existe redução polinomial de SAT para 3-SAT (expansão de cláusulas longas).

(c) "Retornar o menor tour" é um problema de **otimização**

Não é problema de decisão (não responde SIM/NÃO) — não se classifica diretamente em P ou NP.
A versão de decisão associada — "existe tour com custo $\leq L$?" — é NP-completa.

Ponto-chave

Classificar corretamente exige distinguir decisão vs. otimização e identificar a classe do problema de decisão associado.

Gabarito — Exercício 3: Partição $\in$ *NP* e NP-Completude

(a) Certificado e verificador

Certificado: $S_1 = \{5, 3\}$. Tamanho: $O(n)$.

Verificador:

1. Para cada $s \in S_1$: verifique $s \in S$. Custo: $O(n)$.
2. $\sum S = 16$; $\sum S_1 = 5 + 3 = 8 = \frac{16}{2}$. **OK**. Custo: $O(n)$.

Custo total: $O(n) \Rightarrow$ **PARTIÇÃO** $\in NP$. ■

(b) PARTIÇÃO como caso especial de SUBSET-SUM

PARTIÇÃO(S) $\equiv$ SUBSET-SUM(S, $\frac{\sum S}{2}$). O alvo k é simplesmente metade da soma total de S.

(c) Conclusão

SUBSET-SUM $\leq_p$ PARTIÇÃO (dado) e PARTIÇÃO $\in NP$.
Como SUBSET-SUM é NP-completo e se reduz a PARTIÇÃO, concluímos que **PARTIÇÃO é NP-completo**. ■

Resumo

1. **NP-completo:** os problemas mais difíceis de NP. Resolver qualquer um em tempo polinomial implicaria $P = NP$.
2. **Prova de NP-completude:** mostre $L \in NP$ e exiba redução $A_{\text{NPC}} \leq_p L$.
3. **Redução:** transformação polinomial que preserva SIM/NÃO. A **direção** e os **dois sentidos** da equivalência são obrigatórios.
4. **Equivalências:** CLIQUE $\leftrightarrow$ INDEPENDENTE, CICLO-HAM $\leq_p$ TSP... Problemas aparentemente distintos podem ser igualmente difíceis.
5. **Na prática:** NP-completo orienta a escolha de heurísticas, aproximações e solvers.

Para fixar

Domine **certificado / verificador** (prova que $L \in NP$) e o **sentido da redução** (prova NP-completude). Esses dois conceitos formam o núcleo de P vs. NP.

Pergunta para reflexão

Se você descobrisse um algoritmo polinomial para 3-SAT, qual área da computação seria impactada primeiro — e por quê?